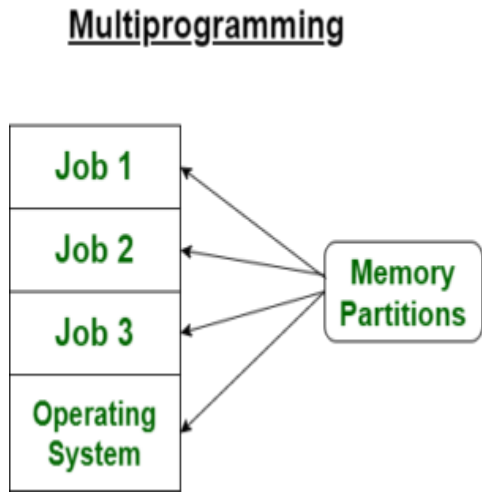
Operating System Structure & Operations



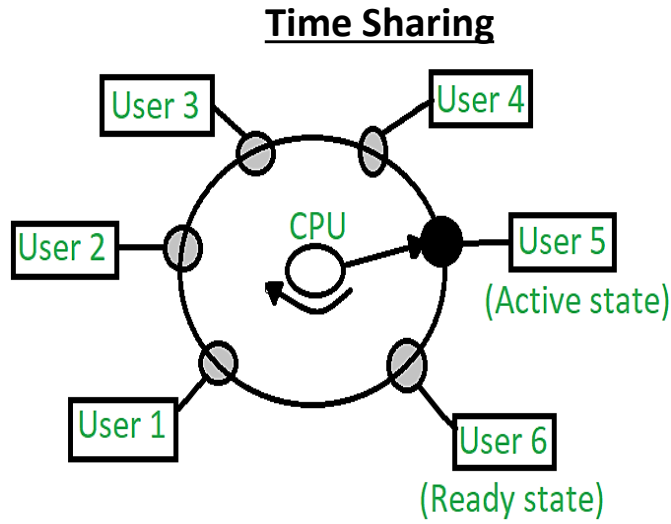
By:

Dr. H. K. Azad
Assistant Professor (Sr.)
SCOPE
Email: hiteshwarkumar.azad@vit.ac.in

Operating System Structure



- Multiprogramming needed for efficiency
  - Single user cannot keep CPU and I/O devices busy at all times
  - Multiprogramming organizes jobs (code and data) so, CPU always has one to execute
  - A subset of total jobs in system is kept in memory
  - One job selected and run via job scheduling
  - When it has to wait (for I/O for example), OS switches to another job
- Timesharing (multitasking) is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating interactive computing
  - Response time should be < 1 second
  - Each user has at least one program executing in memory ⇨process
  - If several jobs ready to run at the same time ⇨ CPU scheduling
  - If processes don't fit in memory, swapping moves them in and out to run
  - Virtual memory allows execution of processes not completely in memory



Operating System functions

- The main functions of operating systems are:

1. Program creation
2. Program execution
3. Input/Output operations
4. Error detection
5. Resource allocation
6. Accounting
7. protection

Operating System Design Issues: Structuring Methods

- The **Structure and Architecture** of an Operating System (OS) refers to how the OS is internally organized.

📈 Why Structure Matters

- A well-structured OS:
- Makes it easier to **add features** and **fix bugs**
- Allows for **better security** and **fault isolation**
- Improves **portability** and **performance tuning**

The main design approaches:

1. Monolithic Architecture
2. Layered Architecture
3. Microkernel Architecture
4. Modular Architecture
5. Hybrid Architecture

Monolithic Architecture

➤ **Description:**
- All OS services (e.g., file system, memory management, device drivers) run in **kernel mode** in a single large process.
- All components can directly call each other and share memory.

➤ **Example:**
- **Linux**, early **UNIX**

➤ **Pros:**
- High performance due to direct communication between components.
- Simple to implement for small systems.

➤ **Cons:**
- Difficult to maintain and debug.
- One bug can crash the entire system.
- Poor modularity.

Layered Architecture

➤ **Description:**
- OS is divided into a hierarchy of layers, each built on top of lower ones.
- Each layer only interacts with its neighboring layers.

➤ **Example:**
- **THE operating system** (early experimental OS)

➤ **Pros:**
- Clear separation of concerns; easier to debug and maintain.
- Improved security and modularity.

➤ **Cons:**
- Performance overhead due to abstraction layers.
- Can be inflexible—lower layers may need to expose too much functionality.

Microkernel Architecture

➤ **Description:**

- Only essential services (like IPC, basic scheduling, and low-level memory management) run in the kernel.
- Other services (file systems, device drivers, etc.) run in **user space**.

➤ **Example:**

- **MINIX**, **QNX**, **macOS (partially)**

➤ **Pros:**

- High modularity and better fault isolation.
- A bug in a user-space service doesn’t crash the whole system.

➤ **Cons:**

- IPC between user-space and kernel-space adds overhead.
- More complex to implement efficiently.

Modular Architecture

➤ **Description:**

- Kernel is structured as a set of loadable modules.
- Combines performance of monolithic design with flexibility of microkernel.

➤ **Example:**

- **Modern Linux kernel** (with loadable kernel modules)

➤ **Pros:**

- Can load/unload parts of the OS at runtime (e.g., drivers).
- Easier updates and better flexibility.

➤ **Cons:**

- Slight overhead due to abstraction.
- Needs careful interface design between modules.

Hybrid Architecture

➤ **Description:**

- A mix of monolithic and microkernel designs.
- Core services run in kernel mode, others may run in user mode.

➤ **Example:**

- **Windows NT**, **macOS (XNU kernel)**

➤ **Pros:**

- Balances performance and modularity.
- Can take advantage of both design philosophies.

➤ **Cons:**

- Increased complexity.
- May not fully achieve the benefits of either approach.

Comparison Table

Architecture	Performance	Modularity	Fault Isolation	Complexity
Monolithic	High	Low	Low	Low
Layered	Moderate	High	Moderate	Moderate
Microkernel	Low	High	High	High
Modular	High	High	Moderate	Moderate
Hybrid	Moderate	Moderate	Moderate	High

Abstractions in Operating Systems

- **Making Complexity Manageable, Enabling Control**
- Operating systems use **abstractions** to simplify interaction with hardware.
- They **hide complex, low-level hardware operations** behind clean, user-friendly interfaces.
- This abstraction layer allows developers to:
  - Write software **without needing to understand** the inner workings of the CPU, memory, disk drives, or network interfaces.
  - **Focus on logic and functionality**, not hardware management.
- Abstractions are **essential to OS design**, enabling:
  - **Portability** across hardware platforms
  - **Resource protection and sharing**
  - **Efficient system management**

What are Abstractions?

Hiding Complexity

- Operating Systems act as a layer between hardware and applications.
- **Abstractions** are simplified representations of complex underlying hardware or software components.
- They hide intricate low-level details from users and programmers.

Why Abstractions?

- **Simplification:** Easier development for programmers.
- **Portability:** Applications run across different hardware.
- **Protection:** Isolation between processes.
- **Resource Management:** OS handles resource allocation behind the scenes.

Key Abstraction: The Process

Process: A Running Program

- An abstraction of a program in execution.
- The OS creates the illusion that each process has its own dedicated CPU.

What a Process Includes:

- **Code (Text Section):** The program's instructions.
- **Data Section:** Global and static variables.
- **Heap:** Dynamically allocated memory.
- **Stack:** Local variables, function calls.
- **CPU Registers & Program Counter:** Current execution state.

Key Abstraction: Virtual Memory

Virtual Memory: Illusion of Infinite RAM

- An abstraction of the computer's main memory (RAM).
- Each process gets its own **virtual address space** (a contiguous range of addresses).
- The OS maps these virtual addresses to physical RAM addresses.

Benefits:

- **Memory Protection:** Isolates processes, preventing one from corrupting another's memory.
- **Memory Expansion:** Allows programs to use more memory than physically available by swapping to disk.
- **Simplified Programming:** Programmers don't manage physical addresses.

Key Abstraction: The File

File: Persistent Storage

- An abstraction of persistent data stored on devices like hard drives or SSDs.
- Hides the physical details of disk sectors, tracks, and cylinders.
- Presents data as a logical collection of bytes.

File System Operations:

- Create, Read, Write, Delete files.
- Organize files into directories/folders.
- Manage access permissions.

Key Abstraction: Devices

Devices: Standardized Hardware Access

- An abstraction of hardware components (I/O devices) like:
  - Keyboard, Mouse
  - Printers, Monitors
  - Network Cards, USB devices
- The OS provides a uniform interface for applications to interact with diverse hardware.

Device Drivers:

- Specialized software that translates generic OS requests into specific hardware commands.
- Allows the OS to communicate with different device models.

Key Abstraction: Threads

Threads: Lightweight Processes

- A unit of execution within a process.
- Multiple threads can exist within a single process.
- Threads within the same process share resources (memory, open files).

Benefits:

- **Concurrency:** Allows a single program to perform multiple tasks simultaneously.
- **Responsiveness:** Keeps an application responsive while performing long-running operations.
- **Resource Sharing:** Efficient use of process resources.

Summary

Abstractions: The Core of OS Design

- **Processes:** Running programs.
- **Virtual Memory:** Isolated and expanded memory.
- **Files:** Organized persistent data.
- **Devices:** Uniform hardware interaction.
- **Threads:** Concurrent execution within a process.

Outcome:

- Simplified programming.
- Enhanced system stability and security.
- Efficient resource utilization



What is the main purpose of abstraction in an operating system?

A) Increase hardware complexity

C) Simplify software installation

B) Eliminate the need for memory management

D) Hide hardware complexity from users and applications

Next



What does thread abstraction provide in an operating system?

A) Lightweight concurrent execution within a process

C) Network security

B) Multi-user support

D) File storage

Next



Device abstraction in an OS helps by:

A) Making devices obsolete

C) Allowing only one type of device to work

B) Removing device drivers from the system

D) Providing a uniform interface for different devices

Next



What abstraction allows programs to use more memory than physically available?

A) Device abstraction

C) Process abstraction

B) File abstraction

D) Virtual memory

Next



Which of the following is an example of process abstraction?

A) Virtual memory

C) CPU scheduling

B) Threads

D) A running program treated as an independent entity

Well done !

Computer System Operations: I/O Structure and Interrupt Handling

Concurrent Execution

- CPU and I/O devices operate **in parallel**, overlapping computation and I/O.
- Improves **CPU utilization** and overall **system throughput**.

Device Controllers

- Each I/O device has a **dedicated controller** (hardware).
- Controller contains:
  - Control & status registers
  - Local buffer
- Acts as an **interface between the device and the OS**.

Local Buffers

- Temporary storage inside the controller.
- Helps **match speed differences** between fast CPU/memory and slow I/O devices.
- Enables **block-based transfers** instead of byte-by-byte transfers.

Computer System Operations: I/O Structure

Memory Transfers

- CPU initiates I/O by issuing commands to the controller.
- Data transfer paths:
  - **Programmed I/O (PIO):** CPU copies data between memory and controller buffer.
  - **DMA (Direct Memory Access):** Controller transfers data directly to/from memory with minimal CPU involvement.

I/O Flow

- Typical data path:

  Device ↔ Controller Buffer ↔ Main Memory ↔ CPU
- CPU **never communicates directly with the device hardware**.

Computer System Operations: Interrupt Handling

Interrupts

- An **interrupt** is a signal sent to the CPU by hardware or software.
- It **temporarily stops the current execution** of the CPU.
- The CPU transfers control to a special routine called the **Interrupt Service Routine (ISR)**.
- Interrupts are used to **notify the CPU about events**, such as:
  - Completion of an I/O operation
  - Hardware errors
  - Timer expiration
- After servicing the interrupt, the CPU **resumes the interrupted program**.

Why Interrupts Are Important

- Enable **asynchronous I/O**
- Improve **CPU efficiency**
- Eliminate **busy waiting (polling)**

Computer System Operations: Interrupt Handling

Interrupts

- Controller generates an **interrupt** upon:
  - I/O completion
  - Error conditions
- Interrupt causes:
  - CPU to pause current execution
  - OS to execute an **Interrupt Service Routine (ISR)**

Interrupt Handling

- Steps:
  - Interrupt occurs
  - CPU saves current context
  - Control transferred to ISR
  - OS services the device
  - CPU resumes interrupted task
- Enables **asynchronous I/O** (no busy waiting).

Benefits of Interrupt-Driven I/O

- Eliminates continuous polling
- Improves **efficiency and responsiveness**
- Supports **multiprogramming and multitasking**

Common Functions of Interrupts

- **Transfer Control to ISR**
  - When an interrupt occurs, control is transferred to the appropriate **Interrupt Service Routine (ISR)** using the **interrupt vector**.
  - The interrupt vector stores the **addresses of all ISRs**.
- **Save Processor State**
  - The interrupt mechanism saves the **address of the interrupted instruction** and CPU state so execution can resume correctly after servicing.
- **Handle Exceptions and Traps**
  - A **trap or exception** is a **software-generated interrupt**.
  - Caused by:
    - Errors (e.g., divide by zero)
    - System calls or user requests
- **Support Asynchronous Events**
  - Allows the CPU to respond to **external events** like I/O completion, timer expiration, or hardware faults.
- **Enable Interrupt-Driven OS**
  - Modern operating systems are **interrupt driven**.
  - The OS remains idle until an interrupt occurs, improving efficiency.
- **One-Line Summary**

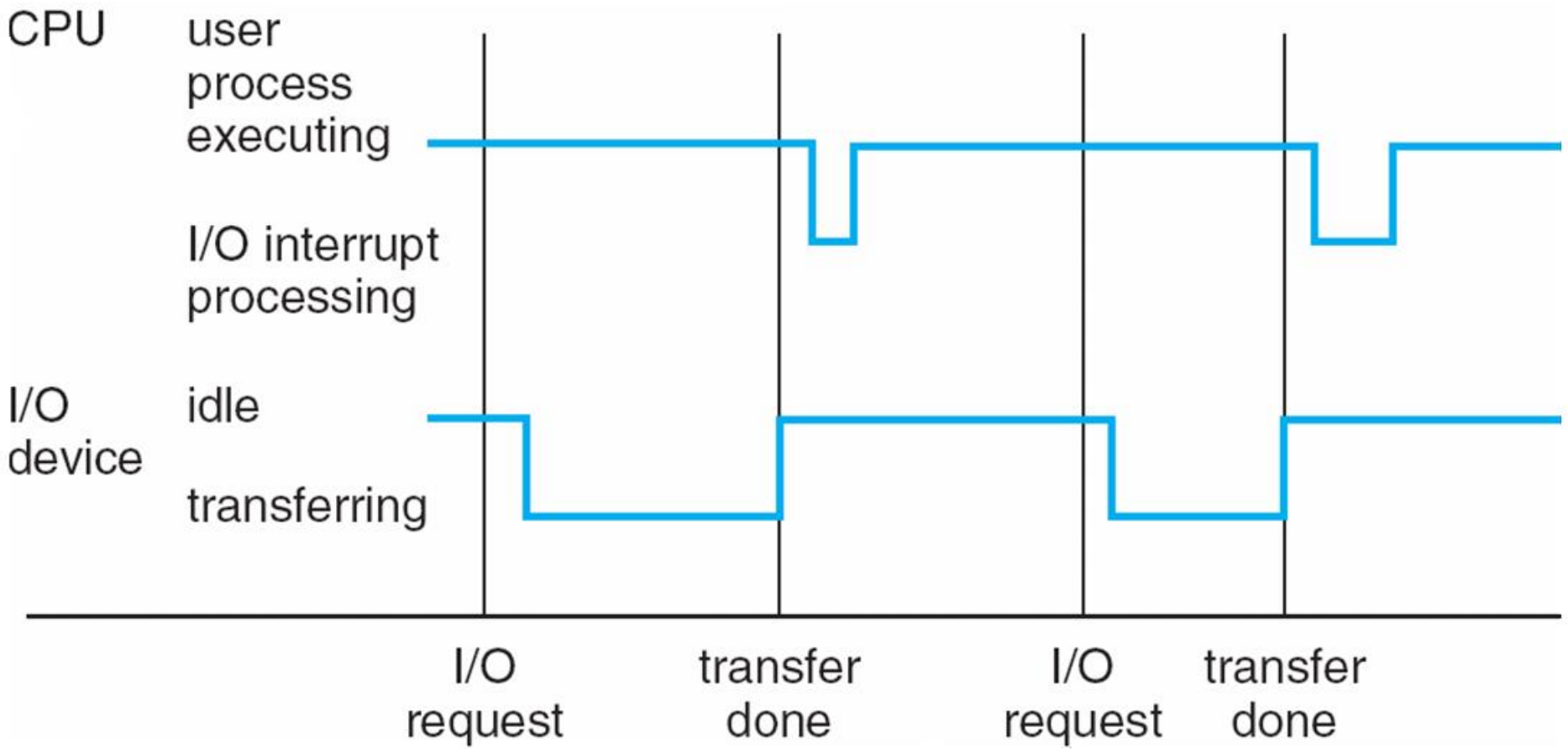
Interrupts allow the CPU and OS to respond efficiently to hardware and software events.

Interrupt Handling

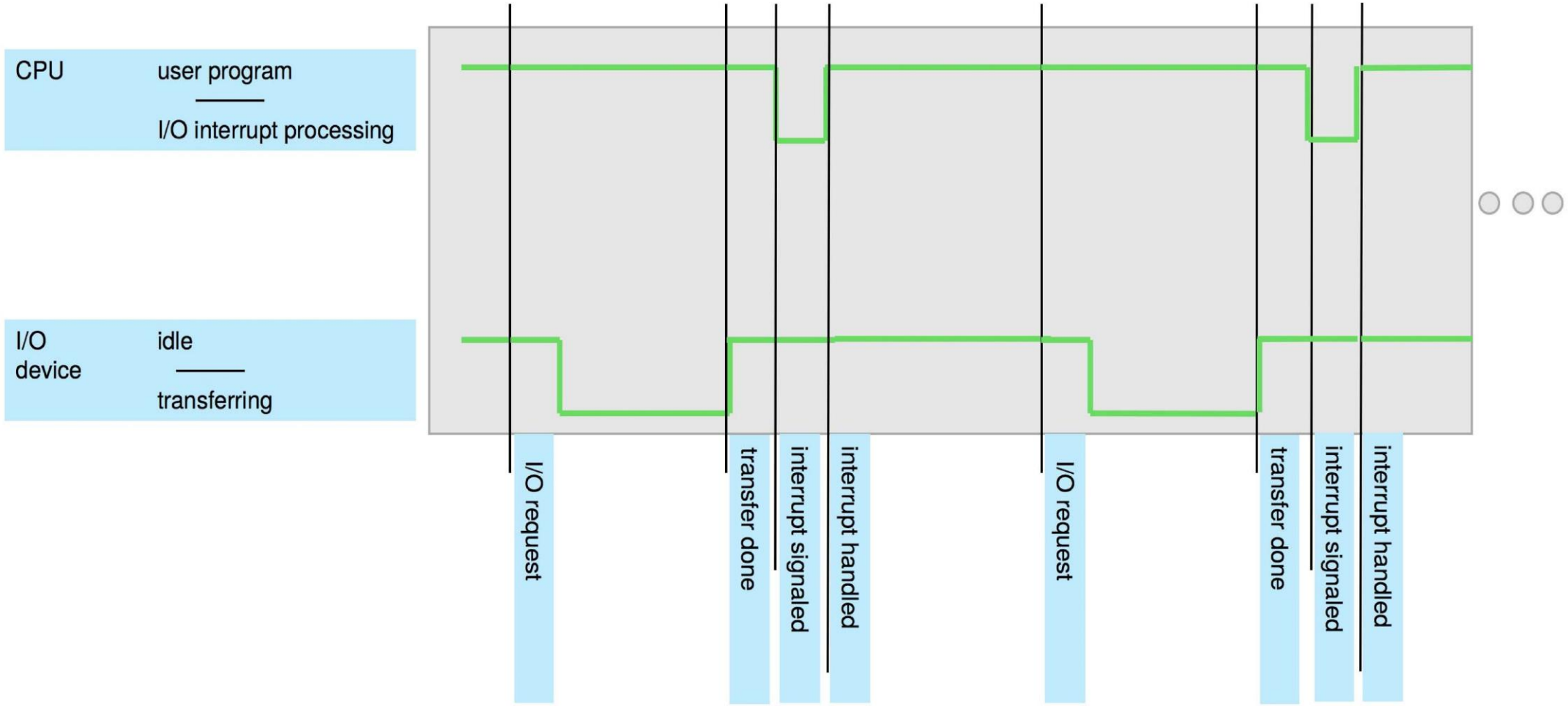
- **Save CPU State**
  - When an interrupt occurs, the operating system **saves the current CPU state**:
    - Program Counter (PC)
    - CPU registers
  - Ensures correct resumption of the interrupted program.
- **Identify the Interrupt**
  - The OS determines **which interrupt has occurred** using:
    - **Polling** – CPU checks devices one by one
    - **Vectored Interrupt System** – interrupt vector gives direct access to the ISR
- **Execute Interrupt Service Routine (ISR)**
  - Control is transferred to the **appropriate ISR**.
  - The ISR performs the required action (e.g., I/O completion, error handling).
- **Restore CPU State**
  - After servicing, the saved CPU state is restored.
  - Execution continues from the point where it was interrupted.
- **Separate Handling Logic**
  - Different **interrupt types** (I/O, timer, exception) have **separate code segments** to handle specific actions.
- **Flow Summary**

  Save state → Identify interrupt → Execute ISR → Restore state → Resume execution

Interrupt Timeline



Interrupt Timeline



I/O Structure

Synchronous (Blocking) I/O

- After an I/O operation starts, **control returns to the user program only after I/O completion**.
- The CPU may:
  - Execute a **wait instruction** (CPU idles until an interrupt occurs)
  - Execute a **busy wait loop** (wastes CPU cycles and causes memory contention)
- Limitations:
  - **Only one I/O request** can be active at a time
  - No overlap of computation and I/O
  - Poor CPU utilization

Asynchronous (Non-Blocking) I/O

- After I/O starts, **control returns immediately to the user program**.
- I/O proceeds **in parallel** with CPU execution.
- User can later check or wait for completion using a **system call**.

Operating System Support for I/O

- **System Calls**
  - Allow user programs to request I/O services
  - Provide mechanisms to wait for or poll I/O completion
- **Device-Status Table**
  - Maintained by the OS
  - Contains an entry for each I/O device, including:
    - Device type
    - Device address
    - Current status (busy, idle, error)
- **Interrupt Handling**
  - OS indexes the **device-status table** upon receiving an interrupt
  - Updates device state and wakes up waiting processes

Key Advantages of Asynchronous I/O

- Supports **simultaneous I/O operations**
- Improves **CPU utilization**
- Enables **multiprogramming and multitasking**

- **One-Line Summary**

I/O structure defines how the OS manages I/O operations using blocking or non-blocking mechanisms with interrupt support.

Storage Definitions and Notation

Basic Units

- **Bit**
  - Smallest unit of storage
  - Can store **0 or 1**
- **Byte**
  - **8 bits**
  - Smallest addressable unit on most computers

Binary Storage Units (Used by Operating Systems)

- **1 Kilobyte (KB)** = 2^{10} bytes = **1,024 bytes**
- **1 Megabyte (MB)** = 2^{20} bytes = **1,024 KB**
- **1 Gigabyte (GB)** = 2^{30} bytes = **1,024 MB**
- **1 Terabyte (TB)** = 2^{40} bytes = **1,024 GB**
- **1 Petabyte (PB)** = 2^{50} bytes = **1,024 TB**
- **1 Exabyte (EB)** = 2^{60} bytes = **1,024 PB**

Standard Unit Relationships

- **1 Byte = 8 bits**
- Each higher unit = **1,024 × previous unit**

Important Note (Optional for PPT / Exam)

- Operating systems use **binary-based units (powers of 2)**.
- Storage manufacturers may use **decimal units** (1 KB = 1,000 bytes), which explains size differences.

One-Line Summary

Computer storage is measured in binary units, where each level is 1,024 times the previous unit.

Storage Structure

Main Memory (Primary Storage)

- The **only large storage** directly accessible by the CPU
- Characteristics:
  - **Random access**
  - **Fast access time**
  - **Volatile** (data lost on power-off)
- Stores:
  - Currently executing programs
  - OS kernel and active data

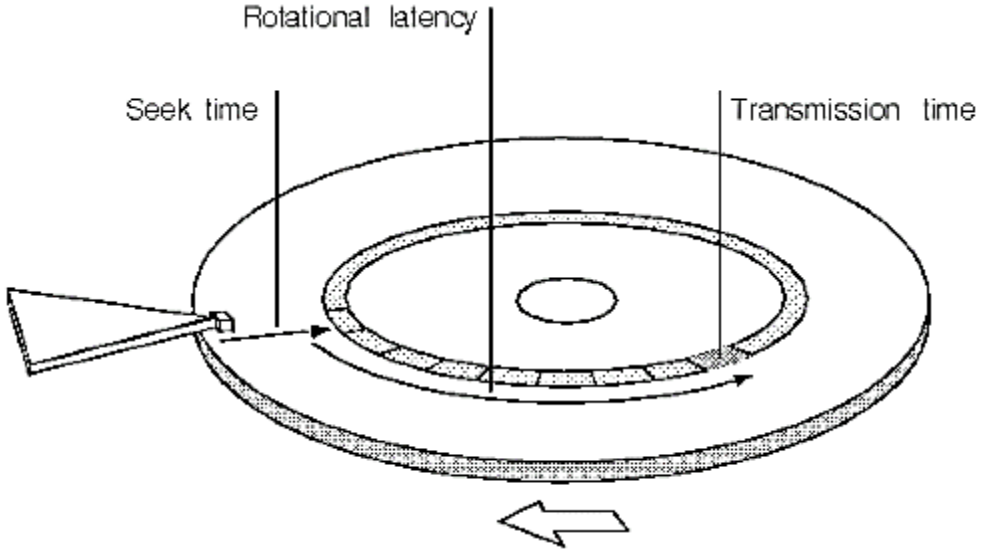
Secondary Storage

- Extends main memory with **large, nonvolatile storage**
- Used for:
  - Permanent data storage
  - Backup and file systems
- Slower than main memory but **much larger and cheaper**

Storage Structure (Cont.)

Hard Disk Drives (HDDs)

- Use **magnetic storage** on rotating metal or glass platters
- Disk organization:
  - Platters → **Tracks** → **Sectors**
- Performance components:
  - **Seek time** – time to move the head to the correct track
  - **Rotational latency** – wait for the sector to rotate under the head
  - **Transfer time** – time to move data
- **Disk controller**
  - Manages communication between disk and system
  - Translates logical requests to physical operations



Solid-State Drives (SSDs)

- Use **flash memory** (no moving parts)
- Characteristics:
  - **Much faster** than HDDs
  - **Nonvolatile**
  - Lower power consumption
- Technologies:
  - SATA SSDs
  - NVMe / PCIe SSDs
- Increasingly **replacing HDDs** in modern systems

Storage Hierarchy

Organization of Storage Systems

- Storage systems are organized in a **hierarchical structure** based on:
  - **Speed** (access time)
  - **Cost per bit**
  - **Volatility**

As we move **up the hierarchy**:

- Faster access
- Higher cost
- Smaller capacity

As we move **down the hierarchy**:

- Slower access
- Lower cost
- Larger capacity

Typical Storage Hierarchy (Top → Bottom)

- CPU Registers
- Cache Memory
- Main Memory (RAM)
- Secondary Storage (SSD / HDD)
- Tertiary Storage (Optical disks, Magnetic tapes)

Caching

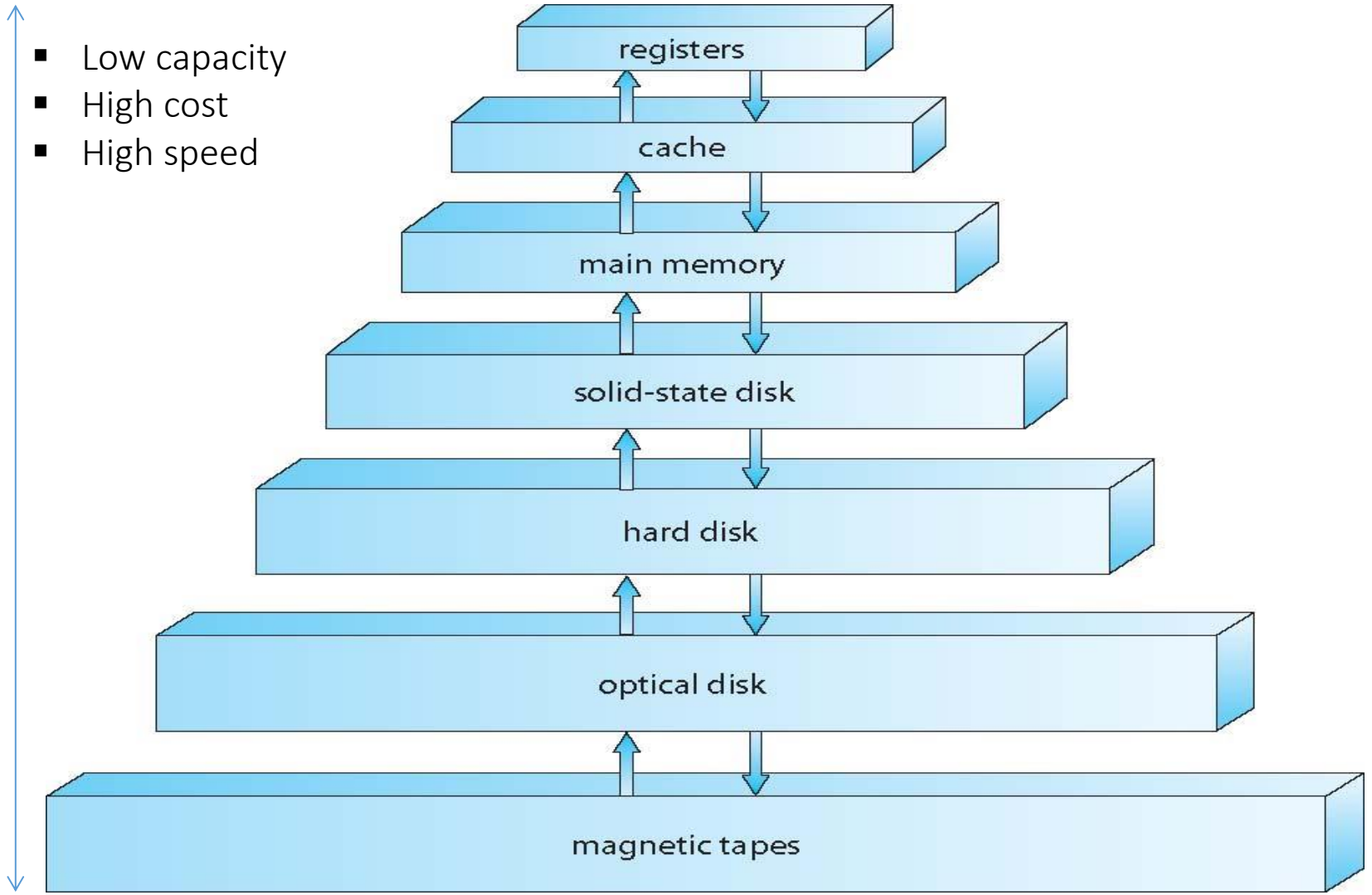
- **Caching** is the process of copying frequently accessed data into **faster storage**.
- Improves system performance by reducing access time.
- **Main memory acts as a cache** for secondary storage.
- **Cache memory acts as a cache** for main memory.

Device Drivers

- A **device driver** is provided for each device controller.
- Responsibilities:
  - Manages I/O operations for the device
  - Provides a **uniform interface** between hardware controllers and the OS kernel
- Allows the OS to remain **hardware independent**.

Storage-Device Hierarchy

- Low capacity
- High cost
- High speed



Caching

- **Caching** is a fundamental performance optimization technique.
- Implemented at multiple levels:
  - **Hardware** (CPU caches)
  - **Operating System** (buffer cache, page cache)
  - **Software** (web/application caches)

Basic Concept

- Frequently used data is **temporarily copied** from **slower storage** to **faster storage** (cache).
- The cache is **checked first** whenever data is requested.

Cache Access

- **Cache Hit**
  - Requested data is found in the cache
  - Data is accessed **quickly**
- **Cache Miss**
  - Requested data is not in the cache
  - Data is fetched from slower storage
  - A copy is placed in the cache for future use

Cache Characteristics

- Cache size is **much smaller** than the storage it caches.
- Cache management is a **key design challenge**.

Cache Design Issues

- **Cache Size**
- **Replacement Policy**
  - Determines which data to remove when cache is full
  - Examples: LRU, FIFO, Random

One-Line Summary

- Caching improves performance by keeping frequently accessed data in faster storage.

Direct Memory Access (DMA) Structure

Purpose

- Used for **high-speed I/O devices** capable of transferring data at speeds close to main memory.
- Reduces CPU overhead during large data transfers.

How DMA Works

- The **CPU initializes the DMA controller** by providing:
  - Memory address
  - Transfer size
  - Direction (read/write)
- The **DMA controller transfers data blocks directly** between:
  - Device buffer ↔ Main memory
- CPU is **not involved** during the actual data transfer.

Advantages of DMA

- Frees the CPU to perform other tasks
- Improves overall **system performance**
- Essential for large, continuous data transfers (e.g., disks, network interfaces)

One-Line Summary

DMA allows high-speed devices to transfer data directly to memory with minimal CPU involvement.

Operating-System Operations

Interrupt-Driven System

- Modern operating systems are **interrupt driven** (hardware and software).
- The OS remains idle until an event occurs, improving efficiency.

Types of Interrupts

- **Hardware Interrupts**
  - Generated by **I/O devices** (keyboard, disk, network, timer).
  - Used to signal events such as:
    - I/O completion
    - Hardware failure
    - Timer expiration
- **Software Interrupts (Traps / Exceptions)**
  - Generated by software execution.
  - Occur due to:
    - **Errors** (e.g., division by zero, invalid memory access)
    - **System calls** (request for OS services)
    - **Process faults** (infinite loops, illegal operations)

Operating-System Operations (cont.)

Dual-Mode Operation

- Enables the OS to **protect itself and system resources**.
- Two execution modes:
  - **User Mode** – normal application execution
  - **Kernel Mode** – OS and privileged operations

Mode Bit

- A **hardware-provided mode bit** indicates the current mode.
- Helps distinguish between:
  - User code
  - Kernel (OS) code
- Ensures security and stability.

Privileged Instructions

- Certain instructions are **restricted** to kernel mode only:
  - I/O operations
  - Memory management
  - CPU control instructions
- Prevents user programs from damaging the system.

System Calls

- Provide a controlled way for user programs to:
  - Request OS services
- A system call:
  - Switches CPU from **user mode → kernel mode**
  - After execution, control returns to **user mode**

Multi-Mode Operations

- Modern CPUs support **more than two modes**.
- Example:
  - **Virtual Machine Manager (VMM) mode**
  - Allows safe execution of **guest virtual machines**

QUIZ TIME!



START THE QUIZ

In a time-sharing system, the CPU:

A) Is dedicated to one process until it finishes

C) Executes multiple processes simultaneously

B) Switches between tasks only on completion

D) Switches between processes frequently

Next

Which of the following allows user interaction with the OS?

A) Shell

C) Kernel

B) Device drivers

D) Firmware

Next

The faster the memory:

A) The larger its capacity

C) The cheaper it is

B) The more power it consumes

D) The closer it is to the CPU

Next

Which of the following represents the correct order in the storage hierarchy from fastest to slowest?

A) Main memory → Cache → Registers → Secondary storage

C) Secondary storage → Main memory → Cache → Registers

B) Cache → Main memory → Secondary storage → Registers

D) Registers → Cache → Main memory → Secondary storage

Next

What is an interrupt in an operating system?

A) A signal from the CPU to start a process

C) A software routine to execute before termination

B) A command to shut down the system

D) A signal that temporarily halts the CPU's current activities

Next

The term non-volatile means:

A) The storage is temporary

C) Data is lost when power is turned off

B) The storage is in RAM

D) Data is permanently stored and retained without power

Next

What is the term for saving the current CPU state during an interrupt?

A) Interrupt generation

C) Polling

B) Scheduling

D) Context switching

Next

Which of the following best describes a monolithic operating system structure?

A) OS functions are split into independent services communicating via messages

C) Each OS function is in its own layer with defined interfaces

B) Only hardware drivers run in kernel space

D) All OS services run in a single large kernel space

Next

In the layered approach to OS design, layers are structured to:

A) Depend directly on hardware

C) Share data and code freely

B) Combine user and kernel space

D) Build each layer on top of lower-level services with well-defined interfaces

Next

A hybrid kernel combines features from which two OS structuring models?

A) Monolithic and microkernel

C) Layered and networked

B) Threaded and pipelined

D) Virtual and real-time

Next

Which of the following is typically moved to user space in a microkernel design?

A) CPU scheduling

C) Interrupt handling

B) Memory management unit (MMU)

D) File system and device drivers

Next

Which of the following best describes multiprogramming?

A) Multiple users on one system

C) Programs communicating over a network

B) One program running on multiple CPUs

D) Running several programs concurrently by switching among them

Next

The kernel remains in:

A) RAM (main memory)

C) User memory

B) Disk cache

D) ROM

Next

Which of the following best differentiates the kernel from the shell?

A) Shell manages memory; kernel runs commands

C) Kernel manages hardware; shell interacts with users

B) Shell is faster than kernel

D) Kernel is hardware; shell is software

Well done!

Thanks